

Laboratorio de predicciones

→ Villarruel Sanchez David Uriel

→ Crisostomo Garcia Israel

```
#####
#  TU TABLA FINAL      ·  ESTA ES LA CAPTURA QUE TIENES QUE ENTREGAR  #
#####
#  expresion              predijiste              salio
#  -----
1   enteroSinValor        0                      0          ok
2   decimalSinValor      0.0          0.0          ok
3   logicoSinValor       false          false        ok
4   (int) letraSinValor  null          0          FALLO
5   textoSinValor        null          null         ok
6   7 / 2                3.5          3          FALLO
7   7.0 / 2              3.5          3.5         ok
8   (double) 7 / 2       3.0          3.5         FALLO
9   (double) (7 / 2)     3.5          3.0         FALLO
10  7.0 / 0              infinity          Infinity    FALLO
11  0.0 / 0.0            Non              NaN         FALLO
12  7 % 2                1          1          ok
13  10 % 5               0          0          ok
14  -7 % 2               NaN             -1          FALLO
15  7 % 2.5              2          2.0         ok
16  12345 % 10           1          5          FALLO
17  12345 / 10           1234.5        1234        FALLO
18  (int) 3.9            4          3          FALLO
19  (int) -3.9           -3          -3         ok
20  (char) 65            65          A          FALLO
21  (int) 'A'            A          65         FALLO
22  'A' + 1             null          66         FALLO
23  (char) ('A' + 1)    null          B          FALLO
24  (byte) 200          null          -56        FALLO
25  5 > 3               true           true         ok
26  5 == 5.0            false          true         FALLO
27  0.1 + 0.2 == 0.3    0.300000000000000~ false        FALLO
28  true && false        null          false        FALLO
29  true || false       true           true         ok
30  vidasRestantes > 0 && 10 / vida~ 0          false        FALLO
31  0.1 + 0.2           0.3          0.30000000~ FALLO
32  Integer.MAX_VALUE + 1 1              -2147483648 FALLO
33  5 + 3               8.0          8          ok
34  "5" + 3             68          53         FALLO
35  1 + 2 + "3"         6          33         FALLO
36  "1" + 2 + 3         5          123        FALLO
#  -----
ACERTASTE 12 DE 36
ESTAS SON LAS QUE TIENES QUE EXPLICAR EN TU DOCUMENTO:
```

Análisis

4.-

Pensábamos que una variable char sin valor iba a comportarse como null, pero no es así. En Java, un char no puede tener null; cuando es una variable de instancia, su valor inicial es 0. Al convertir ese carácter a int, su valor numérico es 0. Por eso el resultado correcto es 0.

6.-

Esperábamos obtener 3.5 porque matemáticamente esa es la división, pero en este caso los dos valores son enteros. Java hace una división entera y elimina la parte decimal, así que el resultado termina siendo 3.

8.-

Aquí pensamos que primero se haría la división entera, pero el (double) convierte el 7 a decimal antes de realizar la división. Entonces realmente se calcula $7.0 / 2$, y como uno de los valores es double, el resultado conserva los decimales: 3.5.

9.-

Lo que nos confundió fue el orden de las operaciones. Primero se calcula $7 / 2$ como división entera y se obtiene 3; después ese resultado se convierte a double, quedando 3.0. Por eso no aparece 3.5.

10.-

infynity (mi respuesta tenía un error de escritura). Creímos que cualquier división entre cero produciría un error, pero aquí 7.0 es un double. En las operaciones de punto flotante, Java representa la división entre un número distinto de cero y cero como Infinity. Por eso aparece Infinity.

11.-

Pensamos que cualquier división entre cero produciría infinito, pero aquí tanto el numerador como el resultado de la operación corresponden a una división indefinida: $0.0 / 0.0$. En Java, cuando se trabaja con double, este caso se representa como NaN, que significa que el resultado no es un número.

14.-

Nos confundimos pensando que el signo negativo podía hacer que el resultado fuera NaN, pero % simplemente obtiene el residuo de la división. Al dividir -7 entre 2, el residuo es -1, así que ese es el resultado.

15.-Mi predicción fue 2, pero al estar involucrado 2.5, la operación se realiza como double, por lo que Java muestra 2.0. El programa la marcó como correcta, aunque realmente mi predicción estaba mal porque no coincidía con la salida que debía producir.

16.-Pensé que el operador % podía darme el primer número o alguna parte del número, pero en realidad obtiene el residuo de la división. Al dividir 12345 entre 10, el residuo es 5, porque 12340 es divisible entre 10 y quedan 5. Por eso el resultado es 5.

17.-Pensé que la división conservaría el decimal .5, pero los dos valores son enteros. Por eso Java realiza una división entera y elimina la parte decimal. 12345 / 10 da 1234, no 1234.5.

18.-Aquí confundí el casting con un redondeo. Al convertir 3.9 a int, Java no busca el entero más cercano, sino que elimina directamente la parte decimal. Por eso el resultado es 3 y no 4.

20.-Supuse que al hacer el casting se mantendría el número 65, pero char representa caracteres. El valor 65 corresponde a A, así que Java muestra la letra A.

21.-En este caso ocurrió lo contrario. En lugar de conservar la letra, al convertirla a int Java obtiene su valor numérico. La A corresponde al valor 65, por eso aparece 65.

22.-Mi error fue pensar que sumar uno directamente a la letra produciría B. Sin embargo, cuando un char participa en una operación aritmética, Java utiliza su valor numérico. A vale 65, entonces $65 + 1 = 66$.

23.-Primero se realiza la suma utilizando el valor numérico de A, que es 65. Al sumarle 1 se obtiene 66, y después el casting convierte 66 nuevamente en un carácter, que corresponde a B.

24.-No tomé en cuenta el rango que puede manejar un byte. Este tipo solamente puede almacenar valores de -128 a 127, así que 200 queda fuera del rango. Al hacer la conversión ocurre un desbordamiento y el resultado es -56.

26.-Pensé que serían diferentes porque uno está escrito como entero y el otro como decimal, pero == compara sus valores numéricos. Ambos representan exactamente el mismo valor, así que la condición es true.

27.-Matemáticamente esperaba que fuera true, pero los valores double se almacenan con una representación binaria aproximada. Por eso $0.1 + 0.2$ resulta ser un número muy cercano a 0.3, pero no exactamente igual. Al compararlos con ==, da false.

28.-Me equivoqué al pensar que podía producir null, pero && es el operador lógico "y". Para que el resultado sea true, ambas condiciones tienen que ser verdaderas. Como una es false, toda la expresión da false.

30.-Aquí interpreté la expresión como si pudiera devolver un número, pero las comparaciones y los operadores lógicos trabajan con valores booleanos. Como la condición completa no se cumple, Java devuelve false.

31.-Esperaba ver exactamente 0.3, pero nuevamente interviene la forma en que los valores double representan los decimales. Algunos no pueden almacenarse exactamente en binario, así que Java obtiene un valor ligeramente diferente, aunque muy cercano a 0.3.

32.-Supuse que al máximo de un int simplemente se le podía sumar uno, pero ese tipo ya no puede almacenar un valor mayor que 2147483647. Al pasar el límite ocurre un desbordamiento y el valor vuelve al extremo negativo, dando -2147483648.

33.-Mi predicción fue 8.0, pero ambos números son enteros, así que Java realiza una suma de enteros y obtiene 8. El programa la marcó como correcta, pero realmente la predicción estaba mal porque la salida correspondiente es 8, no 8.0.

34.-Pensé que Java convertiría el "5" en un número para poder sumarlo con 3, pero las comillas indican que es un String. Cuando se usa + con un String, Java concatena los valores, por eso el resultado es "53".

35.-Pensé que Java iba a sumar todos los valores como números, pero las operaciones se realizan de izquierda a derecha. Primero $1 + 2$ da 3. Después ese 3 se encuentra con el "3", que es un String, así que se hace concatenación: $3 + "3"$ produce "33".

36.-Lo que no consideré fue que Java evalúa la expresión de izquierda a derecha. Como empieza con un String, primero $"1" + 2$ produce "12" y después $"12" + 3$ produce "123". Por eso no se realiza una suma normal.